

CS790 Assignment 3 Explanation

Title: CS790 Assignment 3 Explanation

Name: Vansil Rakeshkumar Chauhan

24m0847

Explanation: `task_1.py`

- This script is a custom Tor client that builds a **4-hop circuit** using the Tor ControlPort through the **Stem library**.

Main Goal

- This script builds a 4-hop circuit manually and lets me send GET requests through it.

How it Works

- I start by parsing the current consensus using Stem's `controller.get_network_statuses()`, so I'm working with relays that the running Tor instance already knows about.
- I use a simple function, `pick_relays()`, to randomly select relays for each hop:
 - The first one is a Guard
 - The next two are middle relays
 - The last one is an Exit

- I build the circuit using `controller.new_circuit(path=..., await_build=True)` and wait for it to be established.
 - For the GET request, I needed to make sure the traffic actually goes through **this exact circuit**. That's why I use `__LeaveStreamsUnattached` to disable Tor's default stream attachment behavior.
 - I attach streams manually using `controller.attach_stream(stream.id, circuit_id)` within a `STREAM` event listener.
 - I also added a simple HTTP GET request using the `requests` library through the local Tor SOCKS proxy (`127.0.0.1:9050`) to validate the circuit.
 - To monitor the circuit in real time, I used **Nyx**. It's great for verifying if the relays used match the ones selected.
-

Explanation: `task_2.py`

- This is a more complete version of the custom Tor circuit builder, expanded into an interactive CLI app. It builds **n-hop circuits** using relays selected from the **live consensus file**, which I fetch directly from the Tor Project's public metrics server.
-

What I Built

- The app fetches and parses the latest consensus (using `BeautifulSoup`) so the relay data is always fresh.
- It builds circuits manually using `controller.new_circuit(...)`, just like `task_1.py`, but it supports any number of hops (e.g., 4-hop, 5-hop), which can be specified at runtime.

- Relay selection is done based on flags like `Guard`, `Exit`, and bandwidth. I implemented `weighted_choice()` to mimic Tor's bandwidth-weighted selection.
 - There's an interactive loop that lets the user:
 - Create new circuits
 - List existing ones
 - Attach streams manually
 - Send HTTP GET requests via Tor
 - Reset stream configurations
 - Close circuits
 - All stream handling is done via `EventType.STREAM`. I disable Tor's default behavior with `__LeaveStreamsUnattached`, listen for new streams, and attach them to the user-selected circuit.
-